

計算機科学概論

第 2 回

塚田 武志

2025.10.09 千葉大学

今回の目標

論理ゲートで数を扱う方法を学ぶ

ブール算術

- 背景
- 基本となる回路
- 展望

ブール算術

- 背景
- 基本となる回路
- 展望

二進数

計算機では 2 を底にした数の表現である二進数で数を表す

- ブール値が 0, 1 の二値であるため

$$\begin{aligned}(10011)_2 \\ &= 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 19\end{aligned}$$

$$\mathbb{Z}/16\mathbb{Z} = \{[0], [1], \dots, [15]\} = \{[-8], [-7], \dots\}$$

（桁数） = （回路の配線の本数）であるため，回路では桁数を固定

- 8 bit / **16 bit** / 32 bit / 64 bit / 128 bit

二進数の足し算

十進数と同様の**筆算**で計算可能

繰り上がり

$$\begin{array}{r} 0 \quad 0 \quad 0 \quad 1 \\ 1 \quad 0 \quad 0 \quad 1 \\ + \quad 0 \quad 1 \quad 0 \quad 1 \\ \hline 0 \quad 1 \quad 1 \quad 1 \quad 0 \end{array}$$

和

オーバーフロー
(桁あふれ)

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 0 \quad 1 \quad 1 \\ + \quad 0 \quad 1 \quad 1 \quad 1 \\ \hline 1 \quad 0 \quad 0 \quad 1 \quad 0 \end{array}$$

負の数の表現：2の補数

負の数を表すのは，どのようにしたら良いだろうか？

素朴なアイデア 符号と絶対値の組

$$\{+, -\} \times \{0000, 0001, \dots, 1111\}$$

- 直感的ではあるものの，符号での場合分けが面倒

実際の表現方法 **2の補数**

- $-1 \equiv 15 \pmod{16}$ なので -1 の表現を $15 = (1111)_2$ とする
 - 一般に n bit で数表現しているときは $-k$ の表現は $2^n - k$
 - 先頭ビットが 0 のとき正数，1 のときに負数と解する
- 2の補数表現の加算は**符号なし二進数の加算がそのまま使える**

2 の補数表現における計算

符号反転 ビットごとの反転後に 1 を足す

$$-3 \equiv 2^4 - 3 = (2^4 - 1) - 3 + 1 = \underline{(1111)_2 - (0011)_2} + 1$$

ビットごとの反転

和 符号なし二進数の加算

- ただしオーバーフローの判定方法がかわる
 - **正の数**と**正の数**の和が**負の数**になったら, オーバーフロー

$$5 + 7 = (0101)_2 + (0111)_2 = (1100)_2$$

- **負の数**と**負の数**の和が**正の数**になったら, オーバーフロー

$$(-5) + (-7) \equiv (1011)_2 + (1001)_2 = (10100)_2 \equiv (0100)_2$$

算術論理演算回路 (Arithmetic Logical Unit, ALU)

ビット演算・加算・インクリメントなどを行う装置

- 関連する演算もあるので、全部ひとまとめにすると便利！

例 符号付き整数の引き算

$$x - y = x + (-y)$$

和

ビットごとの反転
とインクリメント

ブール算術

- 背景
- 基本となる回路
- 展望

半加算器 (half adder)

1 桁の和を計算する回路

入力 ふたつのビット

出力 和とキャリー (繰り上がり)

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 0 \\ + 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 0 \end{array}$$

全加算器 (full adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 3つのビット

出力 **和** と **キャリー** (繰り上がり)

$$\begin{array}{r} \mathbf{0} \quad 0 \\ + \quad 0 \\ \hline \mathbf{0} \end{array}$$

$$\begin{array}{r} \mathbf{1} \quad 1 \\ + \quad 1 \\ \hline \mathbf{0} \end{array}$$

$$\begin{array}{r} \mathbf{1} \quad 1 \\ + \quad 1 \\ \hline \mathbf{1} \end{array}$$

加算器 (adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 2つの **16 bit** で表現された数

出力 16 bit で表される **和** (オーバーフローの検出はしない)

$$\begin{array}{r} \mathbf{1} \mathbf{1} \mathbf{1} \\ \mathbf{1} \mathbf{0} \mathbf{1} \mathbf{1} \\ + \mathbf{0} \mathbf{1} \mathbf{1} \mathbf{1} \\ \hline \mathbf{1} \mathbf{0} \mathbf{1} \mathbf{0} \end{array}$$

加算器 (adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 2つの **16 bit** で表現された数

出力 16 bit で表される **和** (オーバーフローの検出はしない)

	1	1	1	1	
		1	0	1	1
+	0	1	1	1	
	1	0	0	1	0

半加算器

加算器 (adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 2つの **16 bit** で表現された数

出力 16 bit で表される **和** (オーバーフローの検出はしない)

The diagram illustrates a handwritten addition of two 5-bit numbers. The first number is 11101 and the second is 01111. A carry-in of 1 is shown on the left. The sum is 10010. A red dashed box highlights the 4-bit section (bits 3 to 6) where the two numbers and the carry-in are added, labeled '全加算器' (Full Adder).

1	1	1	1	
	1	0	1	1
+	0	1	1	1
1	0	0	1	0

全加算器

加算器 (adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 2つの **16 bit** で表現された数

出力 16 bit で表される **和** (オーバーフローの検出はしない)

	1	1	1	1	
		1	0	1	1
+	0	1	1	1	
	1	0	0	1	0

全加算器

加算器 (adder)

1 桁の和を計算する回路. **ただし下の桁からの桁上りも入力とする**

入力 2つの **16 bit** で表現された数

出力 16 bit で表される**和** (オーバーフローの検出はしない)

The diagram illustrates a full adder circuit for a 5-bit addition. It shows two input numbers being added, with a carry-in of 1 from the previous stage. The inputs are:

1	1	1	1	
	1	0	1	1
+	0	1	1	1

The output is:

1	0	0	1	0
---	---	---	---	---

The carry-in of 1 is shown in blue, and the output of the first full adder stage is shown in green. A red dashed box highlights the first two bits of the input and the resulting sum, with the label "全加算器" (Full Adder) in red below it.

加算器 (adder)

1 桁の和を計算する回路. ただし **下の桁からの桁上り** も入力とする

入力 2つの **16 bit** で表現された数

出力 16 bit で表される **和** (オーバーフローの検出はしない)

$$\begin{array}{r} \mathbf{1} \mathbf{1} \mathbf{1} \\ \mathbf{1} \mathbf{0} \mathbf{1} \mathbf{1} \\ + \mathbf{0} \mathbf{1} \mathbf{1} \mathbf{1} \\ \hline \mathbf{1} \mathbf{0} \mathbf{1} \mathbf{0} \end{array}$$

全加算器を複数用意して, **キャリーを繋げば作れる**

ブール算術

- 背景
- 基本となる回路
- **展望**

計算効率について

全加算器を素朴に繋いだ加算器は効率が悪い

- 最悪のケースで k ビットの計算に k 段の論理ゲートを経由

$$\begin{array}{r} 0 \quad 1 \quad 1 \quad 1 \\ \quad 0 \quad 1 \quad 1 \quad 1 \\ + \quad 0 \quad 0 \quad 0 \quad 1 \\ \hline 0 \quad 1 \quad 0 \quad 0 \quad 0 \end{array}$$

- ゲートへの入力の変化から出力の変化には時間がかかるので、出力が多数の論理ゲートを経て定まる回路は計算に時間がかかる
- 計算時間の短縮のために「**キャリー先読み**」という手法が用いられる
- 加算はいたるところで行われる操作なので、改良の影響が大きい

ALU 設計：コストとパフォーマンス

ALU の機能設計はコストとパフォーマンスの兼ね合いで決まる

例 掛け算の実装

- ALU の機能として提供
 - ハードウェアなので計算が速い．一方でコストが高く付く
- ALU は足し算だけを提供して，掛け算はソフトウェア (OS) で実装
 - ハードウェアは簡潔でコストが安い．しかし計算は遅い

本講義の ALU は簡潔さを重視して，機能は限定的

- ALU が「乗算」「除算」「浮動小数点演算」をサポートされることも

課題 2

締切：2024/10/31 23:59 (JST)

課題 2

半加算器，全加算器，加算器，インクリメンタ，ALU を HDL で実装せよ

- 各ゲートの仕様は，教科書のサポートサイトの web 処理系にある

加算器およびALU の実装について説明せよ

提出するもの

- すべてのHDLによる実装をまとめたもの
- 加算機と ALU の実装について説明するレポート

来週の講義もこの課題に取り組みます

参考：学科の Mac のログイン方法

ユーザ名

学生証番号

パスワード

Moodle のパスワード

- 学科 Mac のパスワードが Moodle パスワードと一致しているのはシステムによって同一であることが保証されているのではなく1年生のときの設定時の「一緒にするのがお勧め」という指示のため
- **Moodle パスワードを初期設定のあとで変更した人は学科 Mac のログインは過去のパスワードも試してみてください**
- パスワードが分からない場合はリセットしますので、教えてください

参考：web の処理系の使い方

- web の処理系は Mac の Safari だと動かないかも？
- 学科 Mac には chrome も入っているので、そちらを使ってみては？